A Resistive RAM-Based FPGA Architecture Equipped With Efficient Programming Circuitry

Behnam Khaleghi and Hossein Asadi, *Senior Member, IEEE*

***Abstract*— Despite the considerable effort has been put on the application of *Non-Volatile Memories* (NVMs) in *Field-Programmable Gate Arrays* FPGAs, previously suggested designs are not mature enough to substitute the state of-the-art SRAM-based counterparts mainly due to the inefficient building blocks and/or the overhead of programming structure which can impair their potential benefits. In this paper, we present a *Resistive Random Access Memory* RRAM-based FPGA architecture employing efficient *Switch Box* (SB) and *Look-Up Table* (LUT) designs with programming circuitry integrated in both SB and LUT designs that creates area and power efficient programmable components while precluding performance overhead to these blocks. In addition, we present an efficient scheme to load the configuration bitstream into the memory elements, which makes the configuration time comparable to that of SRAM-based FPGAs. Besides, we investigate the correct functionality and reliability of the programming structure subject to fluctuations in attributes of RRAM cells. Using *Versatile Place and Route* (VTR) tool with the obtained characteristics of the proposed blocks demonstrate that the average area and delay of the proposed FPGA architecture are 59.4% and 20.1% less than conventional SRAM-based FPGAs. Compared with a recent RRAM-based architecture, the proposed architecture improves the area and power by 49.7% and 33.8% while keeps the delay intact.**

***Index Terms*— Emerging non-volatile memory, resistive random access memory, field-programmable gate arrays, programming circuitry, resistive fluctuation.**

I. Introduction

DESPITE their appealing features, *Field-Programmable Gate Arrays* (FPGAs) still suffer from significant power and area overhead that obstructs widespread use of these devices. The overhead mainly arises from the configuration memory bits [1]-[4] that contribute to more than 40% of FPGA total area [5], [6]. In addition to power and area overhead, in which the latter also leads to performance degradation by elongating the wires, the configuration memories of FPGAs also make it inherently susceptible to soft errors [7], [8] and voltage fluctuations [9] which endanger the design reliability, particularly in the deep nano-CMOS era.

To address the area and power issues of FPGAs, several studies have attempted to employ emerging *Non-Volatile Memories* (NVMs), particularly *Resistive Random Access Memory* (RRAM), as a replacement candidate for SRAM-based configuration memory. RRAM is one of the most promising NVM technologies that has attracted intense attention due to its unique advantages [10]. Structurally, RRAM is a two-terminal device comprised of an oxide layer placed between two metal electrodes, composing a metal-insulator-metal structure. By taking advantage of intensified (i.e., detectable) resistance switching of oxide materials, RRAM establishes resistive switching between a *High Resistance State* (HRS) and a *Low Resistance State* (LRS) through applying an appropriate programming voltage. Switching from HRS to LRS is called as *set* process and switching from LRS to HRS is named as *reset* process. Using materials compatible with CMOS *Back-End-Of-Line* (BEOL) fabrication process allows RRAM to be stacked on between top metal layers of device. Such integration provides excellent CMOS compatibility and high density along with zero-leakage nature of these devices.

Integration of RRAM atop CMOS can help reduce the area taken by configuration cells in FPGAs, resulting in power-efficient, smaller, and thereby faster FPGAs [5], [13]. In addition, RRAM provides superior features such as fast write operation, small cell area, and higher density as compared to other NVM counterparts such as *Phase Change Memory* (PCM) or *Spin-Transfer Torque Magnetic RAM* (STT-MRAM) [10]. On the other hand, compared with *Static RAM* (SRAM) and *Dynamic RAM* (DRAM), the relatively higher write duration and write energy of RRAM is much less problematic for FPGAs as compared to cache memory in microprocessors. This is due to the fact that updating the configuration bits of FPGAs is done quite less frequent than memory bits in the cache of microprocessors.

Beside the aforementioned advantages, the resistance state of RRAM cannot be directly utilized in all parts of the FPGA and requires additional circuitry to convert the resistance state to the corresponding voltage level. Particularly, unlike the memory array that can share a sensing circuitry per each column, all of the FPGA configuration cells are continuously being read, demanding efficient sensing peripheral per each cell. In addition, further circuitry is required to enable the write operation for each individual RRAM cell, resulting in larger and less efficient FPGA blocks.

Previous studies on employing RRAM and similar emerging resistive memory technologies in FPGAs range from detailed circuit level to abstract architecture level designs. Several works have designed functional memory elements using RRAM and other emergin technologies that can directly replace SRAM cells in FPGAs [2], [13]–[16]. There are also other works that have proposed various NVM-embedded blocks to come up with more efficient FPGAs [6], [17]–[23].

Manuscript received June 22, 2017; revised October 22, 2017; accepted November 9, 2017. Date of publication January 17, 2018; date of current version May 29, 2018. This paper was recommended by Associate Editor A. Cilardo. *(Corresponding author: Hossein Asadi.)*

The authors are with the Sharif University of Technology, Tehran 11155-8639, Iran (e-mail: behnam_khaleghi@ce.sharif.edu; asadi@sharif.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCSI.2017.2778113

1549-8328 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

These studies suffer from one of the following shortcomings: a) they *only* focus on architectural representation of integrating the non-volatile memories in FPGA and neglect either functionality and/or efficiency of the programming and read circuitries [21], [24], as well as an efficient scheme to load and control the bitstream at the system-level [2], [6], [14], [15], [18]–[20], or b) they propose area and power inefficient SRAM-like configuration cell which also comprises additional programming transistors [2], [14], [25] or *Look-Up Table* (LUT) and *Switch Box* (SB) designs [6], [20]. Moreover, exploiting conventional 2-NMOS/1-Resistor programming structure in these studies, in addition to imposing considerable area overhead, has several major drawbacks as will be detailed in Section II.

In this paper, we present an efficient FPGA architecture employing SB and LUT designs. In the proposed architecture, we design a power-efficient RRAM-based LUT and SB which conforms with conventional FPGA architectures, providing a promising substitute for SRAM-based equivalents. The proposed architecture is equipped with additional circuitry to allow efficient programmability of RRAM cells with minimum area and performance overhead. In the proposed architecture, we add programming paths to the SBs so that all of its internal configuration RRAM cells can be effectively programmed through shared transistors. We have also designed the programming path for a RRAM-based LUT such that the programming takes place with minimum overhead. Finally, we investigate how these two blocks are integrated into the FPGA so that the state-of-the-art FPGA programming circuitries become compatible with the proposed designs. Similar to state-of-the-art flash-based FPGAs, in the proposed architecture we employ conventional CMOS-based flip-flops.

We have exploited HSPICE [26] with physical RRAM model [27] and *Versatile Place and Route* (VPR) tool [28] to examine the proposed designs individually and within an FPGA built upon these designs. HSPICE circuit-level simulations demonstrate that the proposed SB and LUT consume 27.1% and 2.2% less static power compared with their SRAM counterparts. Experimental results over a set of largest *Microelectronics Center of North Carolina* (MCNC) benchmarks [29] show that the overall area and delay of an FPGA using the proposed components are 59.4% and 20.1% less than conventional SRAM-based FPGAs, respectively.

Our novel contributions in this paper are as follows:

(1) We propose an RRAM-based SB design, by leveraging conventional pass-gate based SB, which is equipped with efficient configuration circuitry by sharing programming transistors.

(2) Programmability of the proposed SB is verified by conducting circuit-level SPICE simulations exploiting physical RRAM model, which is neglected in previous studies.

(3) An efficient LUT design using RRAM-based low-leakage voltage-divider configuration cells is proposed. Unlike previous studies [6], [30], the proposed LUT uses only two small programming transistors and is augmented with introduced bypass transistors which allow using intermediate boosting buffers, making it scalable for large inputs.

(4) We demonstrate the integration of proposed designs and associated programming circuitries within the whole FPGA programming scheme. The controlling circuitry as well as the pulse generation scenarios and corresponding timing control are well elaborated.

(5) Detailed evaluation and comparison with the state-of-the-art techniques [2], [6], [14], [22] considering different figure of merits and using realistic RRAM model and parameters are conducted.

(6) The impact of RRAM parameter variation on efficacy of the proposed architecture has been investigated.

The rest of the paper is organized as follows. Section II details previous studies on NVM-based FPGAs. Section III details the proposed SB and LUT architectures. The experimental setup and simulation results have been reported in Section IV. Section V investigates the reliability challenges of the proposed architecture such as impact of parameter variations on its efficiency of the proposed architecture. Finally, Section VI concludes the paper.

II. Related Work

Previous studies either introduce NVM-based configuration cells as a direct substitute for conventional SRAM cells, or propose customized logic and/or routing blocks composed of NVMs. Here we also review the related work that has employed other NVM technologies such as STT-MRAM [14], [20], [22], [23] and PCM [2], [6], [15], [30], [31] since in majority of these works different NVM cells could be analogously utilized.

A. NVM-Based Configuration Cell

A configuration cell comprising two NVM cells embedded in an SRAM-like sensing structure has been proposed in [2], [14], and [25]. While [14] utilizes STT-MRAM cells, [2] and [25] exploit PCM and RRAM cells, respectively. In the proposed cells, NVM cells which are programmed to complementary resistive states, are placed between ground and driver (NMOS) transistors of an SRAM. A so-called sense transistor is also added which connects the Q and $\bar{Q}$ nodes in the case of enabling, i.e., during the device power-on. Releasing the sense transistor causes a race between the nodes and sets them to 0 or 1 based on their relative resistance (NVM) to ground. These cells provide temporal power-down and instantaneous power-on since reconfiguring the same design can be done instantly with enabling the sense signal. However, in addition to an extra (sense) transistor that increases the leakage power of the cell, at least two additional (large) transistors are required to provide programmability, which leads to further area overhead. The overhead becomes worse considering the fact that each RRAM and PCM cell occupies larger area than an SRAM. For instance, the PCM cells used in [2] and [33] occupy $8.75\mu m^2$ footprint which is $30\times$ larger than a typical $140F^2$ SRAM in 45nm technology. Therefore, there is a high chance that the NVM cells to become area bottleneck. Moreover, such NVM-based cells still suffer from the positive-feedback structure of SRAM which is susceptible to soft errors. Functionality of programming structure for the aforementioned cells has not been examined. For instance, [25] uses SRAM internal node to provide the

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

ground voltage. Therefore, the associated transistors should be upsized to tolerate the required current which is neglected in [25].

A non-volatile FPGA employing typical voltage-divider based configuration cells composed of two PCMs as a replacement for SRAM cell has been proposed in [31]. While the cell is reported to improve area and power consumption of LUT and SB, peripheral circuitries and the corresponding scheme required to program the cells are disregarded. Furthermore, a considerable direct current (i.e., $1V$ voltage) is permanently applied on the PCM cells that may lead to a read disturb in PCM-based cells as it has been demonstrated in [35].

A similar PCM based voltage-divider storage element has been proposed in [15] that can replace SRAM cell. The authors report 40% delay improvement achieved by using low resistance PCM (compared with resistance of an NMOS transistor). However, the PCM cells have been employed as configuration cells of the routing multiplexers and produce constant voltages, and hence, they are not in the critical path of the blocks and keep the delay intact. In addition, HRS of such PCM cell with $LRS = 3.7K\Omega$ is estimated to be in the range of Mega-Ohm [6] which consumes a huge leakage power around $1\mu W$. A general stand-alone memory architecture is suggested to program the cells, but important details (e.g., controlling the word- and bit-lines, loading bitstream, etc.) have been neglected.

B. NVM-Based LUT and SB

1) STT-MRAM Based Designs: A STT-MRAM based LUT has been proposed in [22]. The LUT uses conventional *Pass-Gate* (PG) based multiplexer structure in which an *Magnetic Tunnel Junction* (MTJ) cell is placed between ground and the multiplexer branches (i.e., at the leaves of multiplexer tree). Based on the LUT input signals, the corresponding multiplexer branch and its associated MTJ is selected and compared with a base resistance (which has a value between LRS and HRS values) using a sense amplifier (similar to that in Section II-A) at the output of LUT. Relative resistance of the selected MTJ cell and the base resistance determine the output voltage. The drawback of [22] is the sense amplifier which requires a sensing signal to be triggered after each change in LUT inputs. This is analogous to placing a latch at the output of each conventional LUT that demands a clock at each cycle, which prevents cascading the LUTs and/or designing combinational circuits. A similar MRAM-based LUT with duplicated multiplexer sensing structure has been proposed in [23]. This design also suffers from significant area overhead and is vulnerable to resistive fluctuations.

2) PCM-Based Designs: A PCM-based 2×2 SB design employing 24 PCM cells has been proposed in [19]. A major shortcoming of the suggested structure is that some of the PCM cells are not properly placed in between two programming drivers. Therefore, the programming current required for such a cell should pass through another PCM cell which either may be hindered by the cell (if it was previously programmed to HRS) or can cause an unwanted write operation in that cell. In additin, 24 PCM cells occupy huge area with currently matured PCM technology.

PCM-based LUT and SB structures have been proposed in [6]. In the proposed LUT, each SRAM cell is replaced with a single PCM cell and the output is determined by exploiting voltage-dividing between the selected cell and a base resistance, R_{out}, at the LUT output. In such structure, as for [22], only the selected (based on LUT input) PCM cell consumes power. However, the voltage difference between the selected cell and the output node causes a significant power consumption. The SB proposed in [6] is based on a conventional 2×2 crossbar structure in which, similar to [18] and [32], each pass-gate is replaced with an RRAM. The LRS state of the RRAM within the path determines that the path is *on*. Conversely, the HRS RRAM turns off the corresponding path. In order to provide high performance, low LRS PCM cells have been utilized, which also have relatively low HRS due to the limited $\frac{HRS}{LRS}$ ratio of PCM. This causes significant leakage power consumption as detailed in Section III. To provide programmability, input buffers (drivers) of the SB are replaced by multiplexers to choose either the input signal or programming current. Therefore, further area is imposed, especially the multiplexer (added) transistors should be large because they reside in the routing paths.

3) RRAM-Based Designs: Generic RRAM-based multiplexer structure has been proposed in [21]. The proposed structure can be employed in either global (i.e., SB) or local (i.e., intra-cluster) routing network as interconnection building blocks, with any input size. In this study, all multiplexer transistors have been replaced with RRAMs that control the *on/off* state of routing paths by its LRS/HRS state. Since RRAM footprint is currently larger than a typical NMOS transistor, they can become area bottleneck in such a structure. In addition, all RRAM cells in the same multiplexer stage are programmed simultaneously. Thus, the programming transistors (which are stacked in series) should be upsized accordingly in order to tolerate the huge programming current for all RRAM cells and reduce the stacking effect, which leads to significant area overhead.

An FPGA with RRAM-based interconnection networks is suggested in [17] in which only a limited, prefabricated set of buffers are utilized. Therefore, it can improve further CMOS area in conjunction with eliminating the SRAM cells. The proposed SB is similar to [6], [18], and [32], i.e., a 2×2 crossbar in which each pass-gate and SRAM have been replaced by an RRAM. A system-level overview of programming structure has been also discussed in which the RRAMs are arranged as a memory-bank and the cells are selected and programmed consecutively. This individual programming scheme can lead to significant configuration time, especially taking into account the high programming time of RRAM, compared to SRAM. It also uses 2-NMOS/1-Resistor programming structure which has major limitations as explained in [37].

Finally, a one-level RRAM-based multiplexer structure that can be used in routing network of FPGAs has been proposed in [38] and [39]. To resolve the issues with 2-NMOS/1-Resistor programming structure, the authors propose a 4-Transistor/1-Resistor programming structure and use a transistor sharing scheme to reclaim the area. The proposed multiplexer uses a one-by-one programming scheme which excessively increases the FPGA configuration time, particularly, considering the high number of RRAM cells in such an all-RRAM block. Power efficiency of such all-RRAM

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

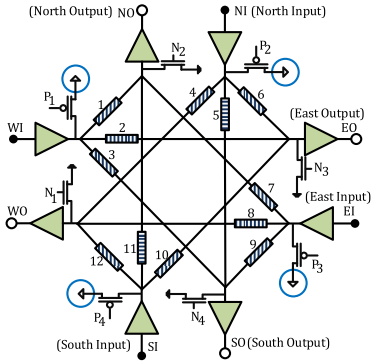


Fig. 1. Proposed RRAM-based SB with the programming peripherals.

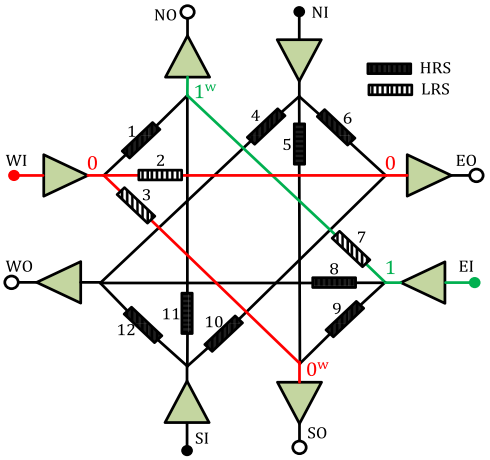


Fig. 2. Examining the functionality of the proposed SB.

structure is the key issue; in an N-input one-level multiplexer, $N-1$ of RRAM cells are configured to HRS and consume power due to the voltage difference between their terminals.

III. Proposed RRAM-Based Architecture

In this section, we first detail the structure of the proposed switch box. Next, we discuss its operation and programming modes and then elaborate how it can be programmed using modern FPGA configuration schemes. Afterwards, we detail the structure and the normal operation and programming modes of the proposed LUT. Finally, we demonstrate how the proposed LUT can be used as a part of the FPGA inside the programming scheme of the state-of-the-art FPGAs.

A. Proposed RRAM-Based Switch Box

Multiplexer and *Pass-Gate based* (PG-based) designs are two commonly used switch box structures (introduced in [36]). PG-based design affords simple architecture and higher performance since only one transistor resides between two input-output terminals. However, it contains higher number of configuration cells, thereby, higher area and power consumption. We borrow the SRAM-based switch box structure of [36] and replace each pass-gate and its associated controlling SRAM cell of the conventional PG-based switch box with a single RRAM cell. There is a high resistance cell in the paths that are supposed to be open, while there is a low resistance cell in the paths that should conduct. Therefore, all SB elements are removed except the buffers, providing an efficient structure. The proposed switch box along with the programming circuitry can be seen in Fig. 1. Similar NVM-based SB structure has been also used in [6], [17], [18], and [32]. Nevertheless, [18] and [32] use two programming transistors for each RRAM (i.e., total of 24 transistors) and [6] change the input drivers which can lead to area or delay overhead (see Section II-B for more details). Our programming structure, however, simply shares eight large transistors between 12 RRAM cells. Substantial impact of programming transistors in SB area will be detailed in Section IV. It should be noticed that the input buffers shown in Fig. 1 are abstract, i.e., they are the output buffers of adjacent SBs. That is, each unidirectional SB has only (four) output drivers.

To illustrate the functionality of the proposed SB, we have depicted the detailed SB architecture in Fig. 2. Input WI drives the output EO through low resistance RRAM2. Since the input terminal of the other two RRAMs connected to EO (i.e., NI through RRAM6 and SI through RRAM10) does not drive any logical value, the strong zero passes properly through RRAM2 from WI. Input EI also drives the output NO by logical 1, through LRS-configured RRAM7. However, NO is also driven by WI, through the HRS-configured RRAM1. Therefore, a voltage dividing takes place in the input of the NO buffer, as $V_{NO} = \frac{HRS}{HRS+LRS} \simeq 1$. Thus, a weak logical 1 (denoted by 1^{w}) passes to the NO buffer. A similar scenario occurs for the SO buffer but in this case, the LRS RRAM is driven by logical 0, hence, $V_{SO} = \frac{LRS}{HRS+LRS} \simeq 0$. Apparently, in the case that EI and WI drive the same logical value, all of the mentioned nodes will be driven by strong 0 or 1.

We have designed a novel programming circuitry and adapted the FPGA programming scheme to fit the proposed SB. The proposed switch box is programmed by the transistors labelled P1 to P4 and N1 to N4, as shown in Fig. 1. The programming voltage is applied to either P1 to P4 transistors while the ground signal is enabled by N1 to N4 transistors depending on the intended cell. The selection of each combination of P1 to P4 and N1 to N4 creates the necessary path for programming of each RRAM cell. For instance, in order to program RRAM7, the transistors P3 and N2 are enabled. Notice that in each write operation, only one P-N pair is enabled; hence the drain node (encircled in Fig. 1) of all P_i transistors is connected to a unique programming node to provide efficient overall configuration loading, as it will be discussed in Section III-D.

We verify the functionality of the programming structure using the RRAM model proposed in [27]. Consider that all RRAM cells of the SB in Fig. 1 have been initially in their HRS state. We validate the functionality by programming (i.e., *set*) RRAM1 to LRS which connects the WI input to NO output. Since RRAM7 and RRAM11 are also connected to NO output (they connect the EI and SI to NO, respectively) we should also check whether they are still in HRS state. That is, they have not been undesirably programmed while programming R1. As it is demonstrated in Fig. 3, P1 and N2 are enabled by applying $V_{prog} = 1.3V$ to their gate voltage, and at the same time, the programming voltage V1 is applied to the source node of P1. Note that source node

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

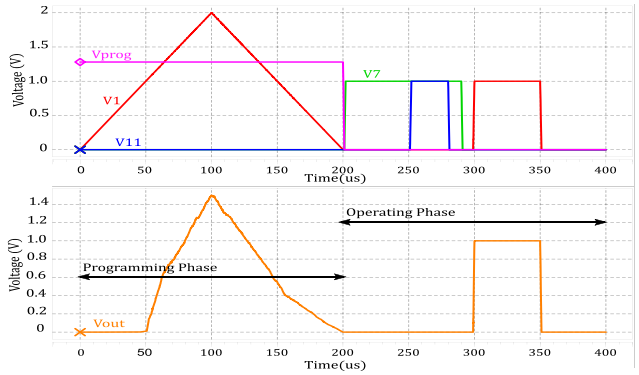


Fig. 3. Verifying the proposed programming structure of SB (top figure shows the applied voltages and the bottom figure shows the output voltage).

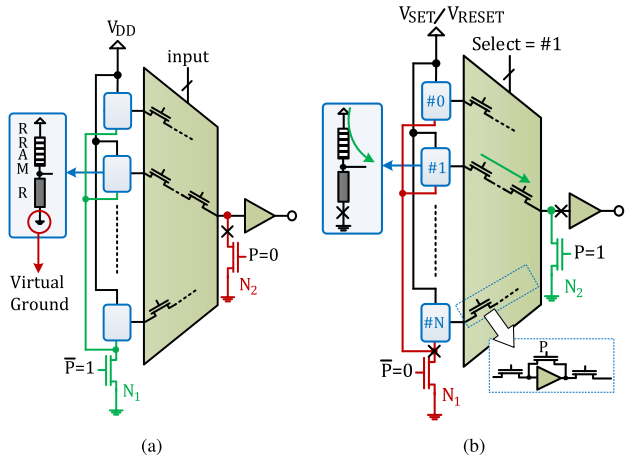


Fig. 4. Proposed LUT in different operating modes. (a) Normal operation. (b) Programming mode.

of all PMOS transistors are connected to the same node, but since only P1 is enabled, the programming current is expected to flow through P1. After $200\mu s$, the programming phase finishes and V_{prog} is cut-off. Right after, three pulses V1, V7 and V11 are simultaneously applied on, respectively, RRAM1, RRAM7, and RRAM11 through the corresponding input buffers (i.e., WI, EI, and SI). The output node (NO) is initially low and rises to one only when V1 rises (at $T = 300\mu s$). Rising V7 and V11, however, does not affect the output voltage which indicates that their path is correctly off. Probable programming reliability challenges have been addressed in Section V-C.

It should be noted that during programming RRAM1, while only N2 is enabled, the programming current generated through P1 can also partially flow through other *leaky paths* to sink the ground using N2 (for instance, through the paths RRAM2→RRAM10→RRAM11 or RRAM3→RRAM9→RRAM7). Similar situation can occur in other programming scenarios which may aggressively *set* some of RRAM cells to LRS and create short paths. In addition, the already programmed (LRS) RRAMs may be *reset* back to the HRS state during the programming of subsequent cells. We examined the possibility of such issues by thoroughly investigating all programming scenarios (beginning with all RRAMs in the HRS state) and measuring the voltage across RRAM terminals. For a $V_{set} = 2.0V$ of the employed RRAMs (see Section IV-A for more details), a maximum voltage of $10^{-5}V$ on LRS and $1.2V$ on HRS cells is observed. Compared to $V_{reset} = 1.3V$, the voltage of $10^{-5}V$ is negligible to *reset* the already LRS cells. In addition, the $\Delta V = 1.2V$ on the terminals of HRS cells is smaller than $V_{set} = 2.0V$ (and even less than $V_{reset} = 1.3V$) to unwillingly *set* those cells. Analogously, during the erase operation (i.e., *reset* all RRAMs to HRS) of each RRAM using $V_{reset} = 1.3V$, the voltage across other RRAMs does not exceed $0.78V$. Therefore, no short paths (which need a voltage even higher than applied $1.3V$) can be created unintentionally.

B. Proposed RRAM-Based Look-Up Table

The proposed LUT cell along with the programming circuitry can be seen in Fig. 4. In our proposed LUT, the conventional SRAM cell is replaced by RRAM-resistor cells that act as a voltage-divider. In contrary with majority of similar work such as [13] and [31], only one programmable RRAM is used in the proposed cell. The passive resistor has a fixed value between RRAM low and high states, so the RRAM-resistor pair provides either an output voltage of zero or one depending on the RRAM value. Based on the cell structure (i.e., RRAM and passive R locations) shown in Fig. 4, to provide logical 1, RRAM should be programmed to LRS, resulting in $V = \frac{R}{R+LRS} \gg 0.5$. Similarly, a HRS RRAM provides $V = \frac{R}{R+HRS} \ll 0.5$.

While the RRAM and passive resistor locations are interchangeable in the proposed cell, we intentionally connect the RRAM to V_{DD} to provide more power saving opportunity as it will be discussed in Section IV-D. Although a voltage-divider scheme consumes leakage power, it is significantly smaller compared with leakage of an SRAM due to using high resistance RRAM cells. The multiplexer and the input signals are used to select the correct memory value as in SRAM-based LUTs. This value is then propagated to the output of the multiplexer through the transmission or pass-gate tree structure.

In the normal operation mode of the proposed LUT, $\bar{P} = 1$, so N_1 delivers the ground node to all the RRAM-based configuration cells. Due to the high resistance of RRAM and R elements, a minimum size NMOS can tolerate the passing leakage current through all configuration cells. In the programming mode, $\bar{P} = 0$ and N_1 cuts off. Otherwise, the applied programming current/voltage (i.e., V_{set} or V_{reset}) will flow to the ground by passing through all configuration cells. However, $\bar{P} = 0$ provides a theoretically infinite resistance to the cells. On the other hand, the *Select* input provides a ground path to the selected cell (cell #1 in Fig. 4(b)) through N_2. Therefore, the applied programming current only passes through and programs the desired RRAM (due to lower, i.e., non-infinite resistance within its path).

It is noteworthy that to provide voltage boosting and prevent exponential delay increase, the inclusion of intermediate buffers within the transistor tree structure of LUT is inevitable. These buffers, however, will prevent flowing the programming current from RRAM cells to ground through N_2. To resolve this issue, we add a parallel bypass transistor to these buffers,

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

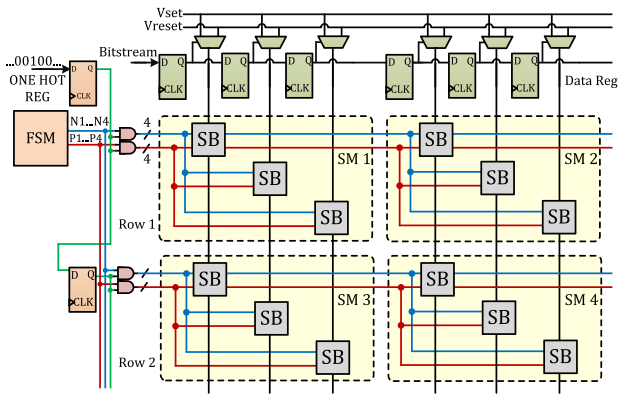


Fig. 5. Integrating the proposed SB in FPGA programming scheme.

as shown in Fig. 4(b). During the programming phase, $P = 1$ enables this transistor (N_2) and allows programming current to flow through it.

C. Discussion

As explained in [37], the 2-NMOS/1-RRAM programming structures suffer from (a) low programming current density due to the voltage drops across transistors, (b) increased transistors threshold, and thereby, reduced driving strength due to the body effect, (c) voltage drops in the driving inverters, and (d) inefficient sizing of transistors for the worst-case current (i.e., *reset* or *set* current). By exploiting unipolar RRAMs, in the proposed 1-PMOS/1-NMOS structure, the NMOS transistors are directly connected to ground (shown in Fig. 1) which eliminates the driving inverters and corresponding voltage drop. In addition, both source and bulk of the NMOS transistors have the same voltage, i.e., $V_s = V_b = GND$; hence, body effect does not occur in NMOS transistors. On the other hand, although in 4-Transistor/1-RRAM structure each pair of PMOS-NMOS transistors is sized according to corresponding *set* and *reset* current, in our 1-PMOS/1-NMOS structure, only one (shared) pair of PMOS-NMOS is used and occupies less area. That is, $P_{max\{set,reset\}} + N_{max\{set,reset\}} < P_{set} + N_{set} + P_{reset} + N_{reset}$. In addition, as detailed in Section IV-A, for our utilized RRAM cell $\bar{V}_{max\{set,reset\}} = 2.0V$ and $I_{max\{set,reset\}} = 100\mu A$ for which a minimum size programming transistor is also adequate for the worst-case scenario. Eventually, as explained in Section IV-A, the shared encircled node of PMOS transistors in an SB should be switched between $V_{reset} = 1.3V$ and $V_{set} = 2.0V$ to *reset* and *set* the RRAM. The required multiplexer is shared between a column of SBs (see Fig. 5), thus it can be upsized by $20\times$ to offset the voltage drop. In addition, since the programming I/O transistors can operate with up to 3.0V and maximum programming voltage is 2.0V, the applied programming voltage can be boosted to conserve the voltage drop.

D. Overall Programming Circuitry

In emerging NVM-based FPGA architectures, loading the configuration bits into NVM cells is one of the major design issues as opposed to the conventional SRAM-based FPGAs, where loading is performed serially [40]. In the proposed architecture, we present a write circuitry to load the configuration bits from registers to the RRAM cells as depicted in Fig. 5. Once the register containing the bitstream data (i.e., *Data Reg*) has been serially loaded, the operation to transfer the data in RRAM configuration cells begins. For switch boxes, a finite state machine generates a sequence to enable the previously explained N1 to N4 and P1 to P4 signals in an appropriate order. At the same time, both *reset* and *set* pulses are sent to their respective lines. Based on the configuration bits, the data register determines which of these pulses are fed into the program lines of each selected switch box. For instance, in the first programming cycle, the FSM enables P1 and N2. Thus, RRAM1 is selected in all switch boxes within a *Switch Matrix* (SM) row, i.e., switch boxes in SM1 and SM2 in Fig. 5. Hence, it takes 12 programming periods (due to the total of 12 cells in the SB) to program all switch boxes in a row. Notice that all P_i transistors within a switch box are connected to a unique programming voltage node, however, only the right one is activated based on the FSM output. Analogously, in the second programming cycle, the FSM enables P1 and N3, so RRAM2 will be selected. Simultaneously, the bitstream frame corresponding to all RRAM2 in the first SM row is loaded to Data Reg. After programming all 12 cells in the first SM row, *One-Hot Reg* selects the second SM row and the same programming scenario repeats.

The proposed programming scheme works also when *set* and *reset* duration are not necessarily the same. Generally, after loading each bitstream frame into the data register which takes $T_{frame} = C \times W/2 \times T_{DFFshift}$ (see Section IV-A), V_{set} and V_{reset} pulses are applied for durations T_{set} and T_{reset}, respectively. Therefore, assuming $T_{set} > T_{reset}$, for the *set* voltage, a simple periodic pulse with the period of $T_{V_{set}} = T_{frame} + T_{set}$ and the duty cycle (i.e., percentage of a cycle in which signal is high) of $D_{set} = \frac{T_{set}}{T_{frame}+T_{set}}$ should be applied. The *reset* pulses are also synchronized with *set* pulses, i.e., $T_{V_{reset}} = T_{frame} + T_{set}$. However, the duty cycle of *reset* voltages is determined by *reset* duration, i.e., $D_{reset} = \frac{T_{reset}}{T_{frame}+T_{set}}$. This means that both *set* and *reset* pulses are disabled (zero) during loading the bitstream and the *set* pulse is active during the remaining programming cycle. Nevertheless, the *reset* pulses become zero after T_{reset}.

The configuration operation for the LUT takes place by consecutively writing each of the RRAM cells. The scheme for this purpose is depicted in Fig. 6. The scheme is similar to that of switch boxes except the path to the ground signal plays the critical role in LUTs. The added transistor before the output buffer can provide the ground signal at the end of the multiplexer tree. This allows having only one of such transistors for each LUT. To program a cell, (a) P is set to active, (b) the appropriate pulse is selected using the serially shifted register (data register), and (c) the path to the newly created ground signal is activated using multiplexer inputs. Notice that all original virtual grounds should be cut off using the $\bar{P}$ transistors in order to avoid undesired write operations. This process has to be repeated for all RRAM resistor pairs in each LUT. Several LUTs can be programmed at the same

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

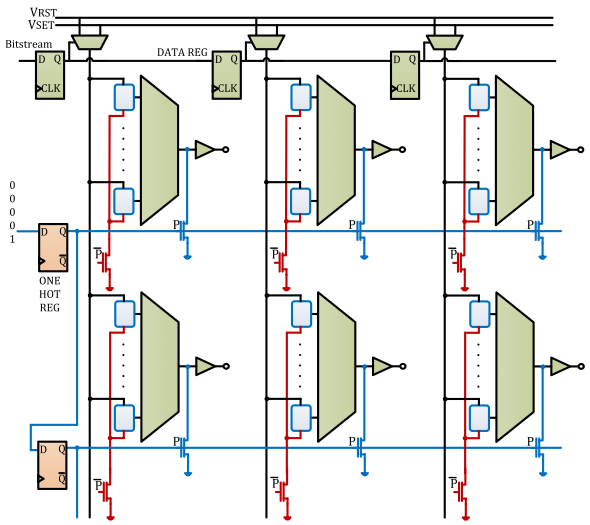


Fig. 6. Integrating the proposed LUT in FPGA programming scheme.

time as long as they are in the same frame and use the data register at the same time.

IV. Evaluation

In this section, we first compare the configuration time of the proposed RRAM-based FPGA with the SRAM-based equivalent by mapping and routing the MCNC benchmarks. Afterwards, we compare the proposed LUT/SB building blocks with state-of-the-art blocks as well as we compare the RRAM-based FPGA built upon the proposed designs with FPGAs based on (a) a STT-MRAM-based non-volatile configuration cell [14], (b) PCM-based non-volatile cells [2], (c) PCM-based LUT and SB designs [6], (d) a *perpendicular Magnetic Tunnel Junction* (pMTJ)-based LUT [22], and (e) a one-level RRAM-based multiplexer for routing network [38].

A. Programming Time

The routing and logic blocks can be programmed simultaneously, however, we examine them separately for the sake of simplicity. We assume that the target device has an island-style architecture as most of the modern FPGAs such as Xilinx Virtex Series [41] and it consists of R rows and C columns of switch matrices and *Configurable Logic Blocks* (CLBs). We also assume the unidirectional routing channel width is W, thus there are $W/2$ switch boxes in each SM.

As detailed in Section III-D, programming each row of SM begins with loading the bitstream frame into Data Reg (see Fig. 5). The total number of flip-flops in Data Reg can be obtained by multiplying the number of SM columns by the number of SBs inside each SM, i.e., $L_{DataReg} = C \times W/2$. RRAM1 in all SBs is simultaneously programmed, which takes $max\{T_{set}, T_{reset}\} = T_{set}$. This procedure repeats for the remaining 11 RRAMs in the first SM row, and then, repeats for the whole R device rows. Accordingly, the total SM programming can be calculated according to Equation 1 in which $T_{DFFshift}$ is the shift delay of each flip-flop (e.g., 0.3ns).

$$T_{SM,prog} = R \times (12 \times (C \times W/2 \times T_{DFFshift} + T_{set})) \quad (1)$$

TABLE I
VPR Architectural Parameters

Parameter	Definition	Value
N	Cluster size	10
K	LUT size	4
I	Routing to cluster input	22
L	Wire segment length	1
Fs	Wire branches in each switch box	3
Fcin	Ratio of channel tracks connected to connection block	0.2
Fcout	Ratio of channel tracks a LB output connects to	0.1
X	Intra-cluster multiplexer size	16

As explained in Section III-A, programming SBs requires to initially erase/*reset* them which requires erase time of $T_{SM,erase}$ as Equation 2.

$$T_{SM,erase} = R \times (12 \times (C \times W/2 \times T_{DFFshift} + T_{reset})) \quad (2)$$

Similarly, supposing there are N_{LUT} number of K-input LUTs inside every CLB (which are arranged in a $R \times C$ array), the total programming time of logic blocks (T_{LB}) can be computed according to Equation 3.

$$T_{LB} = R \times (2^K \times (N \times C \times T_{DFFshift} + T_{set})) \quad (3)$$

To evaluate the programming time, we place and route the 20 largest MCNC benchmarks [29] on minimum size FPGAs using VPR 7.0 toolset [28] to find the array size (i.e., R and C parameters). In the experiments, we have targeted 4-input LUTs, i.e., $K = 4$. The cluster size of $N = 10$ (thereby 10 LUTs within each cluster) has been chosen as it is considered in most recent studies [28]. Notice that the FPGA flow (map, place, and route) has nothing to do with the underlying technology or programming scheme. Therefore, no modification in the CAD tool is required for this purpose. The architectural setup used in VPR is reported in Table I. Unidirectional SBs arranged in a *Subset* topology has been considered. These parameters along with area and timing parameters (see Section IV-B and Section IV-C) are wrapped an architecture file with $X = 16$ (see Table I) provided in VPR power repository.

In our experiments, we use RRAM cells with $1G\Omega$ and $10K\Omega$ HRS and LRS resistances, respectively and small $1\mu m^2$ footprint which has been properly fabricated in [42]. For $10K\Omega$, measured *reset* and *set* currents are $100\mu A$ and $\sim 3\mu A$, respectively. An average 2.0V and 1.3V *set* and *reset* voltages are reported. It should be noticed that RRAM cells necessarily do not show Ohmic behaviour, especially in the HRS state [11], [42]. This means that the programming voltage of RRAM is not simply corresponded to its programming current (and vice versa), i.e., $I \neq \frac{V}{R}$ or $R \neq \frac{V}{I}$. Since the programming speed of the employed RRAM has not been reported, we pessimistically assume $T_{set} = 50ns$ and $T_{reset} = 10ns$ which are large-enough comparing with similar devices [11]. Finally, we set $T_{DFFshift} = 0.24ns$ by synthesizing a shift register using Synopsys Design Compiler [26] and Nangate 45nm Standard Cell Library [43] and – conservatively – choosing minimum size flip-flop cells. Notice that operating frequency of a shift register is independent of its width (i.e., $T_{DFFshift} = T_{clk_to_Q} + T_{setup}$).

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

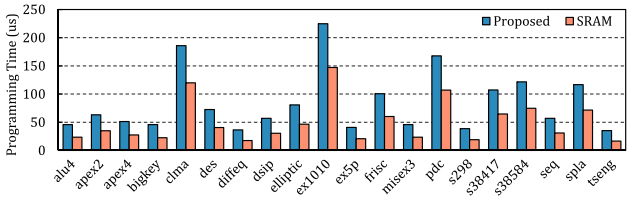


Fig. 7. Comparing the programming time of the proposed RRAM-based FPGA and SRAM-based counterparts.

Fig. 7 compares the programming time (as sum of $T_{SM} = T_{SM,prog} + T_{SM,erase}$ and T_{LB}) of the FPGA based on the proposed RRAM-based LUTs and SBs with the conventional SRAM-based FPGAs. For the SRAM-based FPGA, the total number of configuration bits is calculated by Equation 4 and a maximum programming rate of $371.4MB/s$ (i.e., $\sim 0.337ns$ per SRAM) is assumed [44].

$$N_{SRAM} = (R \times C \times W/2 \times 12) + (R \times C \times N_{LUT} \times 2^K) \tag{4}$$

While the programming time of an individual RRAM cell is substantially higher than an SRAM cell (i.e., $\frac{50ns}{0.337ns} = 148\times$), due to the efficient programming circuitry of the proposed designs, the configuration time is increased by only 81%. This is 83× faster compared with individually programming of RRAM cells (i.e., 50ns per cell) as in [17]. For instance, for *pdc* benchmark with a 20×20 array size and channel width of 106, configuration time of the logic and routing resources will be $31.4\mu s$ and $63.5\mu s + 73.1\mu s$ (erase and program), respectively, with a total of $167.9\mu s$. On the other hand, it contains 318,400 configuration cells which results in $318,400 \times 0.337ns = 107.2\mu s$ and $318,400 \times 50ns = 15,920\mu s$ (i.e., 95× of the proposed method) for, respectively, conventional SRAM-based shifting and RRAM-based one-by-one programming scheme [17]. In larger designs such as *ex1010*, the delay overhead of shifting a frame (into Data Reg) is well compensated by simultaneous programming a large fraction of cells, hence such designs have better relative programming time. For parameters of a moderate-size commercial Virtex-II consisting of 64×64 CLB array, $W = 192$ and $N = 4$ [5], the configuration time in the proposed design is only 39.3% larger than that of the equivalent SRAM-based FPGA.

B. Area

Area of the proposed LUT is reduced by eliminating the SRAM cells and placing the RRAM cells atop CMOS. However, a few extra transistors, including N_1 and N_2 and bypass transistors for intermediate buffers (in overall, eight buffers right after the third stage of LUT) are added. Due to the low programming current ($I_{reset} = 100\mu A$), these transistors can be even selected to have minimum width, i.e., 90nm in the 45nm process. Area of the proposed SB is reduced by removing the SRAM cells and associated pass-gates. However, as shown in Fig. 1, eight programming transistors has been added in the structure of SB. The P_i transistors are directly connected to the programming voltage source and should provide an average 2.0V (up to maximum 2.5V) *set* voltage for the employed RRAMs; thus, larger transistor with thick gate oxide should be utilized. As reported in [37], I/O transistors of a commercial 45nm technology with $\frac{W}{L} = \frac{320nm}{270nm}$ can tolerate V_{GS} and V_{DS} of 2.5V.

TABLE II
VPR EXPERIMENTAL PARAMETERS (AREA IS DIMENSIONLESS AS DETAILED IN SECTION IV-B)

Parameter	Value	Parameter	Value
Switch type	Buffered	SRAM-based SB area	133.7
Segment length	1	Proposed SB area	43.9
SB buffer stage ratio	5×	SRAM-based LUT area	233.5
SRAM area	6.0	Proposed LUT area	137.5
SB pass-gate size	10×	Prog. transistors size	$320nm/270nm$

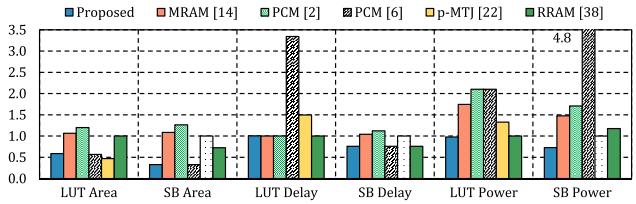


Fig. 8. Comparison of the proposed architecture with the related work in terms of area, delay, and power.

Since the area footprint of a design depends on the geometrical shape of its layout, we use *minimum width transistor* area model which follows the dimensionless formula $Area(x) = 0.447 + 0.128x + 0.391\sqrt{x}$ [45] in which x is the ratio of transistor strength (size) to the minimum width transistor in the same technology. It gives the area as a factor of area of minimum width transistor (which is typically $60L^2$) and has been adjusted for 65nm and smaller technologies. Further details can be found in [45]. Table II summarizes the area parameters used in VPR experiments. Both SRAM- and RRAM-based LUTs employed in the experiments are based on transmission-gates and include isolating buffers (inverter) after the configuration cells to provide higher drive strength and reliability. During programming the proposed LUT, these buffers can be bypassed similar to the scheme explained in Section III-B. Area of programming transistors (assumed as 2× of standard transistors [38]) is included in both the proposed SB and LUT designs.

Fig. 8 compares the area (as well as the delay and power which will be discussed in Sec. IV-C and Sec. IV-D) of the proposed LUT and SB designs with the aforementioned studies. The results reported in this figure will be later used to compare the efficiency of the FPGAs built upon these blocks. All results are normalized to that of the SRAM-based block. Actual quantities of SRAM- and the proposed RRAM-based designs have been represented in Table II. It should be noted that [22] has proposed a NVM-based LUT, but lacks a NVM-based SB. Also, the study in [38] proposes a NVM-based SB and lacks a LUT design. Therefore, for the sake of holistic comparison in the subsequent sections, we assume SRAM-based designs for those lacking circuits. Area of both [2] and [14] has been slightly increased due to larger cell areas (8.9 and 7.0 minimum width transistor area, compared to 6.0 for SRAM). Area of one-level RRAM-based SB [38] is also higher than the area of the proposed SB since [38] employs two programming transistors per each RRAM cell with an additional shared pair. Area of LUT presented in [6]

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

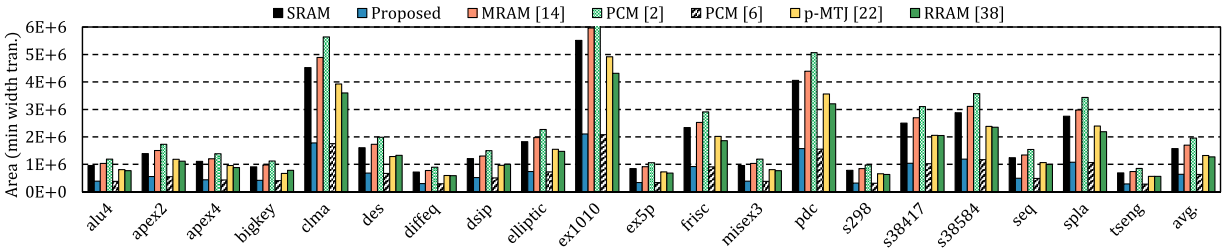


Fig. 9. Comparing the total area between the proposed architecture and related work.

is slightly smaller than our proposed LUT since the proposed LUT comprises a set of bypass transistors for internal buffers, as explained in Section III-B. Eventually, the MTJ-based LUT structure [22] has the minimum area because it eliminates the input buffers.

Fig. 9 compares the area of the proposed RRAM-based FPGA and the related studies over the MCNC benchmarks using the parameters of Table II and Fig. 9. The results reveal that, compared to SRAM-based FPGAs, total area is reduced by 59.4%, on average. This improvement converges with [5] which indicates that if all configuration cells and SBs are moved to top layers, the area can be reduced by ∼57%. Notice that the area of RRAMs has been set to zero since they can be stacked atop CMOS layers. In addition, the area of the RRAM layer is not larger than the CMOS layer (i.e., RRAMs are not bottleneck). The total area of 12 RRAM cell is $12\mu m^2$ [42]. On the other hand, based on the Nangate Cell Library [43], a 5× buffer occupies $\sim 2.5\mu m^2$, therefore the area corresponding to four buffers of the proposed SB is $10\mu m^2$. In addition, the area of the eight programming transistors would be larger than $2\mu m^2$, making the aggregate area of the CMOS part larger than that of RRAM.

Programming Peripheral: The programming circuitry of the proposed architecture comprises $C\times\frac{W}{2}+R$ and $N\times C+R$ flip-flops (including data and one-hot ones, respectively) for SBs and LUTs configuration, respectively (see Fig. 5 and Fig. 6). Also, $C\times\frac{W}{2}+8\times R$ and $N\times C+R$ column and row drivers are required, respectively. For SRAM-based FPGAs, a memory-array-like structure for configuration cells can be considered, as in [40], in which N_{SRAM} number of configuration bits are arranged as a $\sqrt{N_{SRAM}}\times\sqrt{N_{SRAM}}$ array. The number of SRAMs can be obtained from Equation 4. This structure has $\sqrt{N_{SRAM}}+\sqrt{N_{SRAM}}$ (column and row) flip-flops, and $\sqrt{N_{SRAM}}+2\sqrt{N_{SRAM}}$ (for Wordline and Bitline/$\overline{\text{Bitline}}$) drivers. According to Nangate Open Cell Library, the area of a flip-flop and maximum-size buffer is $5\mu m^2$ and $13\mu m^2$, respectively. Using this information with the C, R, and W values obtained from VPR reveals that the programming peripheral of the proposed architecture, on average, occupies 19.3% less area compared with SRAM-based architecture.

C. Delay

The proposed RRAM-based LUT does not affect the delay since it employs the same multiplexer structure of an SRAM-based LUT. In addition, as for the SRAM-based LUTs, the RRAM-based LUT includes an isolating buffer between the configuration cells and the multiplexer input to provide robustness by preventing rush currents from the multiplexer to the cells (which can cause bit flip in the SRAM cells) and also providing higher drive strength for the multiplexer.

TABLE III
HSPICE DELAY MEASUREMENTS PARAMETERS AND RESULTS

Parameter	Value	Parameter	Value
SRAM routing res.	50.9Ω	SRAM LUT delay (avg)	$102ps$
RRAM routing res.	32.6Ω	RRAM LUT delay (avg)	$103ps$
SRAM routing cap.	$11.8fF$	SRAM SB + wire delay	$99ps$
RRAM routing cap.	$7.6fF$	RRAM SB + wire delay	$75ps$

Delay of the LUT and SB designs is measured by using HSPICE circuit level simulation and employing High-Performance 45nm *Predictive Technology Model* (PTM) [46]. To model the RRAM cells, we exploit RRAM model presented in [27] and change the LRS and HRS values based on the parameters of the employed RRAM [42]. Sizing of switch box buffer and transistor have been reported in Table II. The resistive and capacitive characteristics of metal layers for local and global routing of default 22nm process are first extracted using COFFE tool [45] with the architectural parameters of Table I. Afterwards, we scaled the extracted values to 45nm using the scaling parameters obtained from [47], i.e., $\frac{R_{45nm}}{R_{22nm}}=(\frac{22}{45})^2$ and $\frac{C_{45nm}}{C_{22nm}}=(\frac{45}{22})^{0.15}$. In the experiments, we also take into account the role of area reduction in the wire length and its resistance and capacitance [5]. To this end, we correlate the area and wire length by $l=\sqrt{\frac{\bar{s}}{s}}$ in which l is the wire length scaling/reduction factor and $\bar{s}$ and s are the new and original area, respectively. In other words, if the area is scaled by α, then the device dimensions, and hence, wire lengths and the corresponding resistance and capacitance will be scaled by $\sqrt{\alpha}$. As observed in Section IV-B, RRAM-based FPGA reduces the area by 59.4%, hence, $\alpha=0.41$. It is noteworthy that the exploited RRAM model [27] takes the inherent parasitic capacitance between electrodes of an RRAM cell into account. The parasitic capacitance can be a delay bottleneck in an all-RRAM design such as [38] and [39] wherein all RRAM cells are connected to a single output node.

HSPICE reports for the RRAM- and SRAM-based LUT and SB delay are summarized in Table III. As reported in this table, 24.2% improvement in SB delay has been obtained. This improvement arises from scaled area of RRAM-based FPGA, and thereby, reduced wire resistance and capacitance. It is noteworthy that a minimum width NMOS transistor

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

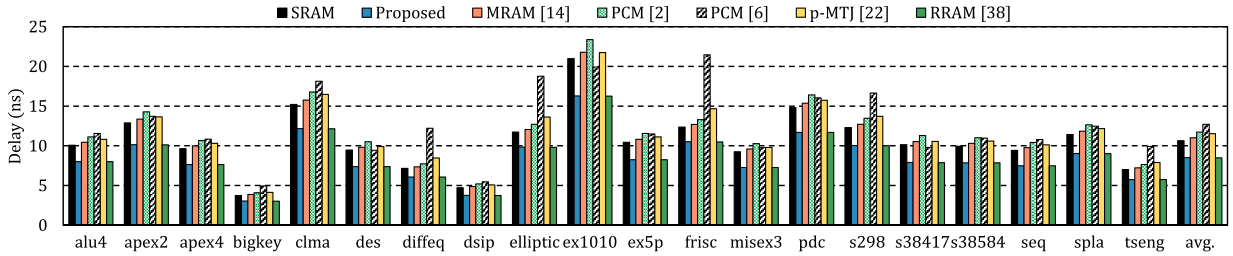


Fig. 10. Comparing the total delay between the proposed architecture and related work.

resistance is $\sim 9K\Omega$ [28]. Since $10\times$ width transistors are used for SRAM-based SB, the resistance of pass-gate would be $\sim 10\times$ smaller than that of employed RRAM cell with $R_{LRS} = 10K\Omega$. Pass-gate based switch, however, considerably increases the delay since the NMOS transistor cannot appropriately pass the logical one. On the other hand, the transmission-gate based SB increases the area significantly. Therefore, to boost the baseline SRAM-based SB, we used SRAM cells with $V_{DD} = 1.2V$. For longer wire segments, e.g., $L = 4$, the delay could further be decreased since in larger segment lengths, the wire delay becomes dominant, which is reduced in the proposed designs. The normalized delay of the previous studies has been provided in Fig. 8. The delay of the proposed LUT, [14], and [2] has almost remained intact with regard to SRAM-based LUT because of using the same multiplexer structure and providing the required voltage level at multiplexer inputs (i.e., configuration cells). Delay of the LUT in [6], however, has been increased because of the passive output-resistance which is used for voltage-dividing and also resides within its critical path. Large HRS of PCM cells significantly increases the delay. On the other hand, similar to the proposed SB, [6] improves the SB delay by replacing low-resistance PCM cells instead of pass-gates and having smaller wire length due to device shrinking. Delay of the pMTJ-based LUT structure is determined by the interval in which the sense signal should be enabled to have a successful read operation. The increase in SB delay of [2] and [14] stems from increased FPGA area, and subsequently, wire delay. Experimental results over the MCNC benchmark are shown in Fig. 10. On average, the proposed design reduces the total delay by 20.1% compared to SRAM-based FPGAs.

D. Power Consumption

1) Cell Structure: Resistive state of the RRAM cell (i.e., HRS or LRS) and the percentage of time in which a cell contains a specific configuration play a substantial role in determining the overall power consumption of the proposed LUTs. Therefore, we first attempt to find the optimum cell structure and the passive resistor value, i.e., R. For this end, analyzing the configuration bits of MCNC benchmarks revealed that about 70% of the LUT configuration cells hold zero and the remaining is configured to one. Table V illustrates the structures and the corresponding leakage power wherein the first structure consumes less power. Notice that $LRS \ll R \ll HRS$, thus $R + HRS$ and $R + LRS$ are simplified to HRS and R, respectively.

2) Optimum R: While specifying R to its maximum allowable value (i.e., $R = HRS/2$ which provides an effective

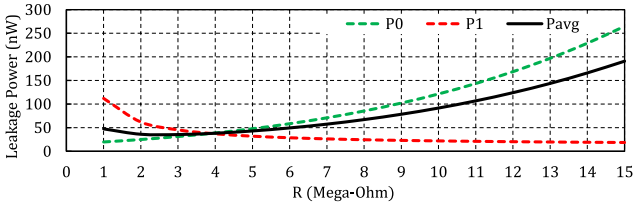


Fig. 11. Calibrating the optimum R value to minimize the cell power.

one and a logical zero $\simeq \frac{1}{3}$) makes the voltage-divider power efficient, it causes the associated isolating buffer to be always *on* which consequently will consume significant power. Indeed, the output logical zero of voltage-divider part should be considerably smaller than the threshold voltage ($V_{th} \simeq 0.3V$) of associated buffer to prohibit significant leakage power. Analogously, logical one of the structure should be larger than $V_{DD} - V_{th}$. Moreover, it provides weak drive strength to the multiplexer tree. Therefore, we carry out HSPICE simulations investigating the whole R space such that $P_{avg} = 0.7\ P_0 + 0.3P_1$ is minimized. As demonstrated in Fig. 11, iterating with $10M\Omega$ steps suggests that $R = 30M\Omega$ provides the best power efficiency. The power includes the power of resistive structure and the associated isolating buffer.

3) Estimation: The power consumption of an RRAM inside the proposed switch box depends on the voltage difference on its terminals. Hence, in order to accurately estimate the static power of each benchmark, we obtain the average signal *probability* of the wires by exploiting ACE 2.0 activity estimator [48]. Accordingly, the probability of voltage difference between the RRAM cells can be obtained. In order to estimate the dynamic power of each benchmark, we first calculate its average signal *activity* using the ACE tool. Next, we conduct HSPICE simulations to obtain the dynamic power of a single switch box using the wire resistance and capacitance represented in Table III. Initially, we assume an activity factor $\alpha = 1$ and frequency of 100MHz, and also assume that one output of the switch boxes is active (i.e., are switching). Table IV summarizes the HSPICE static power results for individual designs. Dynamic power of SRAM- and RRAM-based SBs are, respectively, $1.66\mu W$ and $0.73\mu W$. Note that the dynamic power of logic elements is equal in both the RRAM- and SRAM-based designs since the proposed LUT does not affect the original multiplexer structure and, additionally, the configuration cells have no impact on dynamic power consumption. The reduction in dynamic power of the proposed SB is due to the smaller wire capacitance of the RRAM cells compared with the large pass-gates of the SRAM-based SB.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

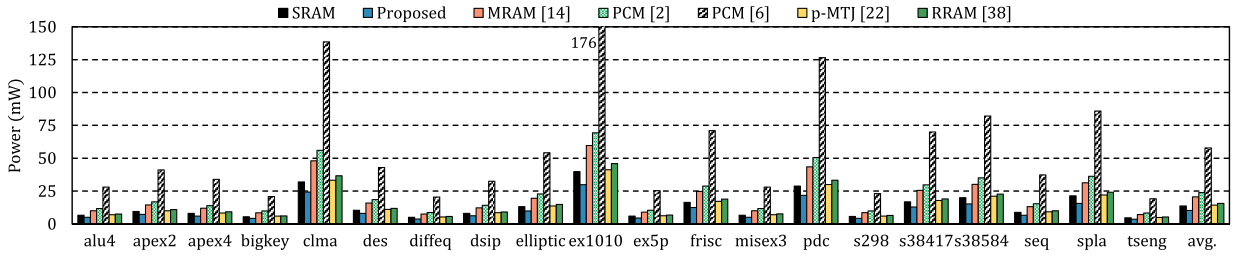


Fig. 12. Comparing the total power between the proposed architecture and related work.

TABLE IV
STATIC POWER PARAMETERS OBTAINED BY HSPICE SIMULATIONS

Design	Power (μW)	Design	Power (μW)
RRAM LUT (all 0)	0.82	SRAM LUT (all 0)	0.87
RRAM LUT (all 1)	1.06	SRAM LUT (all 1)	1.01
RRAM SB (avg)	0.78	SRAM SB	1.07

TABLE V
POWER CONSUMPTION OF DIFFERENT RRAM-BASED CELL STRUCTURES

Cell	Config 0	Config 1	$\sim P_0$	$\sim P_1$	$\sim P_{avg}$
	$\frac{R}{R+HRS}$	$\frac{R}{R+LRS}$	$\frac{1}{HRS}$	$\frac{1}{R}$	$\frac{0.7}{HRS}+\frac{0.3}{R}$
	$\frac{LRS}{LRS+R}$	$\frac{HRS}{HRS+R}$	$\frac{1}{R}$	$\frac{1}{HRS}$	$\frac{0.7}{R}+\frac{0.3}{HRS}$

The relative power consumption of the designs proposed in the related work can also be obtained from Fig. 8. According to this figure, both [2] and [14] increase the LUT and SB power compared with SRAM-based design because of adding the (large) sensing transistor within their SRAM-like sensing circuitry which adds to the static power. On the other hand, experiments revealed that [6] could improve the LUT power (compared with our proposed architecture) by tuning the resistor in its structure. Nevertheless, it would increase the LUT delay intensively as discussed in Section II. Therefore, optimum parameters that minimize its power-delay product are used in our simulations. An advantage of the LUT proposed in [6] is that only the selected cell consumes power. Notice that our results are in contrast with that in the original paper of [2] as this study has reported $1.2nW$ leakage power for the entire 4-input LUT. This value, however, is even smaller than the power of a minimum-size inverter [43]. As for the LUT, the SB power of [2] and [14] is increased due to their increased cell power. The SB power of [6] is increased due to using low resistance PCM cells which consume significant power when a $\Delta V = 1V$ occurs on their terminal. As discussed in Section II, the NVM cells of the LUT proposed in [22] do not consume leakage power. Nonetheless, it requires to trigger the sense signal which floats the SRAM-like sensing circuitry and consumes significant power on that interval.

Fig. 12 demonstrates the overall power consumption of each circuit which is obtained by post-processing the architectural results for each circuit (e.g., array size and channel width) and applying its actual activity factor and segment utilization. An average saving of 24.3% with respect to SRAM-based designs has been achieved over the MCNC benchmarks which stems from reducing the static and dynamic power of the routing resources.

TABLE VI
COMPARING DIFFERENT SRAM- AND RRAM-BASED LUTS

	SRAM/RRAM LUT4	SRAM/RRAM LUT5	SRAM/RRAM LUT6
Delay (ps)	102/103	126/127	139/140
Power (uW)	0.91/0.89	1.75/1.71	3.45/3.37
Area	233.5/137.5	467.1/270.9	932.1/548.1

It should be noted that the impact of the architectural parameters on dynamic power (i.e., F_s, F_{cin} and F_{cout} which are the same for all benchmark circuits) had been inherently considered while obtaining the wires capacitance and resistance using COFFE tool [45]. The channel width, however, does not affect the power of each individual SB. It is because $\frac{I}{4} \times 2$ *Connection Block* (CB) multiplexers with $F_{cin} \times W$ are connected to each horizontal or vertical channel from adjacent CLBs. Thus, totally $\frac{I}{4} \times 2 \times F_{cin} \times W$ inputs are connected to W tracks of the channel, resulting in $\frac{I}{4} \times 2 \times F_{cin}$ inputs per routing track, which induce a capacitance independent of channel width.

E. Impact of LUT Input Size

To demonstrate the scalability of the proposed architecture, we have compared the area and average delay and power of the proposed four to six input LUTs in Table VI. Expectedly, delay and power of SRAM-based and the proposed LUTs are similar in LUTs with different sizes. It is because the original multiplexer-based structure of the LUT has not been modified in the proposed design and the RRAM configuration cells are not in the critical path. In addition, since the average power consumption of the RRAM cell is almost equal to an SRAM cell, both designs consume similar average power. On the other hand, area of a K-input SRAM- and RRAM-based LUT can be estimated as $2^K S_{SRAM} + (2^K - 1)A_{mux2}$ and $(2^K - 1)A_{mux2}$, respectively. Therefore, $\frac{A_{LUT,RRAM}}{A_{LUT,SRAM}} \simeq \frac{2^K A_{mux2}}{2^K A_{SRAM}+2^K A_{mux2}} = \frac{A_{mux2}}{A_{SRAM}+A_{mux2}}$ which is independent of LUT size. According to experimental results in Table VI, area improvement of the proposed LUTs are between 41% and 42%, i.e., it is independent of number of the inputs.

V. RELIABILITY

In this section, we examine the impact of uncertainties in parameters such as programming time and HRS and LRS resistance values, which may hinder the industrial success of RRAM-based devices on the proposed designs. At the end, reliability aspects of the proposed programming structure is discussed.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

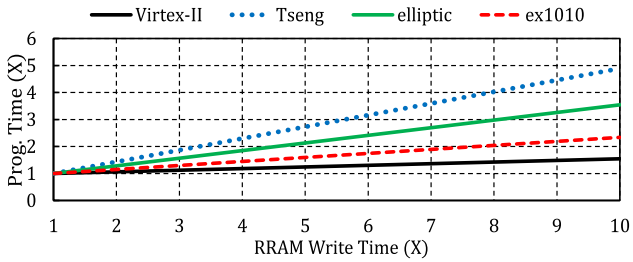


Fig. 13. Programming time of RRAM-based FPGA (with respect to initial time) for T_{set} and T_{reset} from 1× to 10×.

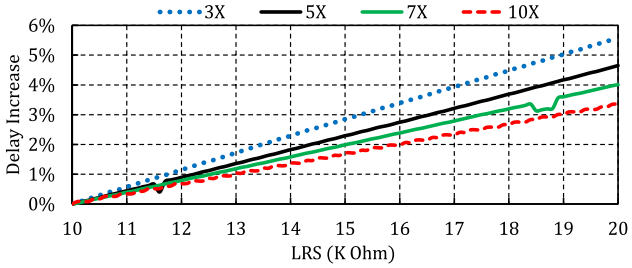


Fig. 14. Delay of SBs with different buffer size due to LRS variation.

A. Programming Duration

The programming speed of the proposed FPGA is determined by both *set* and *reset* operations. As it has been investigated in [49], in the worst case, 10× increase in programming duration of devices with initial T_{set} of ∼100ns (TiO_2) can be observed. It is noteworthy that there is a trade-off between programming duration and cell disturb immunity. Thus, providing long *reset* or *set* periods enhances the programming reliability, in addition to taking the worst-case cells duration into consideration. Therefore, we estimate the programming time of the proposed RRAM-based FPGA considering T_{set} and T_{reset} from 1× (i.e., initial 50ns and 10ns) to 10× (i.e., $0.5\mu s$ and 100ns). Fig. 13 demonstrates the programming time of RRAM-based FPGAs as RRAM write time (*reset* and *set*) increases from 1× to 10×. For this end, four devices with different sizes are selected, i.e., *tseng* (smallest), *elliptic* (medium), and *ex1010* (largest). In addition, the largest Virtex-II device, i.e., XC2V8000 with 93,184 LUTs which can be assumed as a $\sim 96 \times 96$ device is also considered. As shown in Fig. 13, the programming time of larger devices is less affected since in these devices, the T_{set} and T_{reset} factors are prorated by the frame loading time. *Tseng* exhibits 4.9× increase in programming time when T_{set} and T_{reset} increases to 10×, however, this value is only 1.5× for Virtex-II device.

B. HRS and LRS Variation

The device-level phenomenon behind the origin of HRS and LRS resistance variations together with techniques to alleviate them (e.g., by tuning the oxide thickness using additional buffer oxide layer or using write-verify technique) has been discussed in the literature. For more details, the reader can refer to [50]. LRS resistance variation manifests itself as increase in the original value. On the other hand, HRS drift exhibits as decrease in the primary resistance. Such techniques can reduce the distribution of tail bits of HRS to $> 0.5\times$ and the tail bits of LRS to $< 2\times$ of the intended values.

We examine the impact of resistance variations of the RRAM cells in the delay and power characteristics of the proposed designs. To this end, we suppose no device-level or architectural-level enhancements on the RRAM resistance have been carried out; hence a pessimistic 2× increase and 0.1× decrease (i.e., decrease to 0.1× of the initial value) [50] for the LRS and HRS resistance values is considered, respectively. It should be noticed that the delay of the proposed SB merely depends on the LRS value of the RRAM. However, the leakage power of the proposed LUT and SB depends on both the LRS and HRS cells. Thus, a two-dimensional resistance state should be investigated for the latter cases. In addition, since the configuration cells of the LUT are not in its critical path, variation in RRAM resistance does not affect the LUT delay (unless the associated buffer fails to provide a strong 0/1 which does not occur in the considered range of variations).

Fig. 14 represents the increase in SB delay (with different buffer strengths) as the LRS increases from original $10K\Omega$ to the maximum of $20K\Omega$. This 2× increase results in only 4.7% delay increase in the SB used in our experiments (i.e., SB with 5× buffer size) because the main contributor in the SB delay is the input/output buffer and the corresponding wire segment resistive and capacitive parameters. Therefore, even assuming worst-case LRS value which increases the SB delay to 83 ps, the proposed SB has still smaller delay than SRAM-based counterpart. Extrapolating the results reported in Fig. 14 reveals that up to 6× increase in RRAM LRS value ($60K\Omega$) in the proposed SB can be tolerated, i.e., it still affords a delay smaller than SRAM-based SB delay. In addition, as it is evident from Fig. 14, the impact of RRAM variation diminishes in larger buffers.

Fig. 15(a) and Fig. 15(b), respectively, demonstrate the leakage power of the proposed SB and LUT with regard to varying LRS and HRS values. In both figures, LRS variation has negligible impact on the design power. It is because, as mentioned in Section IV-D, in the proposed SB, LRS cells consume almost zero power since there is no voltage difference between terminals (nodes) of such cells. Additionally, in the proposed LUT, the RRAM is in its LRS state when the cell holds one. Accordingly, the cell power will be equal to $V^2/(LRS + R) \simeq V^2/R$ since R is significantly greater than LRS, i.e., $R \gg LRS$.

As illustrated in Fig. 15(a), the leakage power of the proposed SB increases by 20.6% as the HRS values of all cells reduce to 0.1×. Therefore, the voltage difference on a HRS cell nodes (see Section IV-D or Fig. 2) results in approximately 10× power consumption of such cells.[1] However, low fraction of cells with such condition (i.e., having voltage difference between nodes) and relatively high power of SB large buffers reclaims its total power increase. On the other hand, as shown in Fig. 15(b), drift in RRAMs HRS down to 0.3× imposes up to 90% power overhead and it exceeds 200% when the RRAMs resistance diminishes to 0.2×.[2] The exponential increase of power as the HRS reduces corresponds to leakage

[1]The power of such HRS cells increases from $1/1G\Omega = 1nW$ to $10nW$.
[2]HRS below $300M\Omega$ is not shown in Fig. 15(a) as it disfigures the scales.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

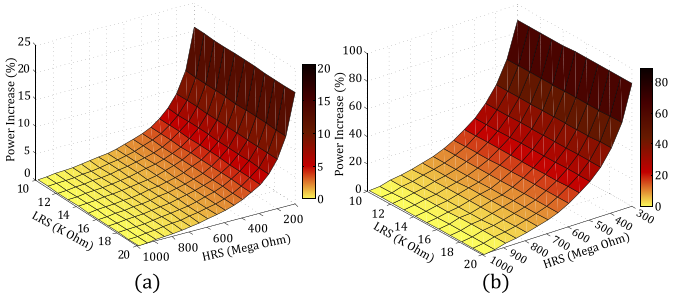


Fig. 15. Static power distribution of the proposed switch box and LUT with respect to HRS and LRS variation. (a) SB power distribution. (b) LUT power distribution.

power of the associated isolating buffers (see Section. IV-C). Low HRS resistance value produces weak zero (e.g., $HRS = 300M\Omega$ results in $V = R/(R + HRS) = 130mV$) which subsequently applies a near-V_{th} voltage to the configuration buffer and pushes its transistors into high leakage saturation region. Nevertheless, this problem could be mitigated by choosing high-V_{th} buffers – especially as the buffers are not in the critical path.

C. Programming Reliability

Back to Fig. 1, during programming (for instance) RRAM1, $P1$ is enabled and delivers the programming voltage/current to the output node of buffer WI. In this condition, depending to its input, either the NMOS or PMOS transistor of WI buffer is *on*. The former scenario causes the programming current to sink to ground through the WI buffer (in addition to N2) and consequently reduces the current density necessary to program RRAM1. To address this issue, the input buffers of the proposed switch box can be equipped with enabling (i.e., tri-state) transistors to cut off the buffer during the programming and preclude of sinking the programming current.

On the other hand, since the programming current flows through the transistor N2, a non-zero voltage occurs on input node of the NO buffer. If this voltage rises above the operating voltage of the transistors of NO buffer, (which is typically $1V - 1.2V$ in 45nm technology) it can cause them to breakdown. Nevertheless, during the reset process, $V_{reset} = 1.3V$ and resistance of the RRAM increases from initial 10 $K\Omega$ (to 1 $G\Omega$). Thus, the resistance of P1-RRAM1 pair will be always larger than that of N2 during the reset process. Thus, voltage of input node of NO buffer will be smaller than $\frac{R_{N2}}{R_{P1}+R_{RRAM1}} \times 1.3V = 0.65V$. Analogously, during the set process, a voltage equal to $2.0V$ will be applied to source node of P1 and RRAM1 resistance begins to reduce from 1 $G\Omega$ downto 10 $K\Omega$. In the worst case (i.e., $R_{RRAM1} = 10\ K\Omega$), a maximum voltage of $\frac{2.0}{2} = 1.0V$ will be applied to NO which is far below the breakdown voltage.

VI. Conclusion

In this paper, we presented switch box and lookup table designs for the RRAM-based FPGAs to integrate the programming circuitry in these blocks and create efficient programmable components. By taking advantage of RRAM appealing features and eliminating the CMOS-compatibility and scalability of flash memory, the proposed RRAM-based FPGA offers the features of SRAM-based FPGAs (e.g., design reconfiguration and short time-to-market) with non-volatility of flash-memory-based FPGAs. Experimental results demonstrated that despite using one programmable RRAM per configuration bit, the proposed architecture increases the configuration time by only 39.3% compared with commercial-size SRAM-based FPGAs while it enhances it by 83× compared to previous studies that use one-by-one programming scheme. Simulation results over the MCNC benchmarks demonstrated that the proposed RRAM-based FPGA reduces the total area and delay by 59.4% and 20.1%, respectively. By eliminating the always-on SRAM-based configuration cells, the static power reduced by 24.3%, on average. Finally, keeping the logic dynamic power intact, the dynamic power of our proposed architecture decreased by 56.0%, mainly due to shrinking the routing wire length.

References

[1] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 26, no. 2, pp. 203–215, Feb. 2007.

[2] K. Huang, Y. Ha, R. Zhao, A. Kumar, and Y. Lian, "A low active leakage and high reliability phase change memory (PCM) based non-volatile FPGA storage element," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 61, no. 9, pp. 2605–2613, Sep. 2014.

[3] S. Yazdanshenas and H. Asadi, "Fine-grained architecture in dark silicon era for SRAM-based reconfigurable devices," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 61, no. 10, pp. 798–802, Oct. 2014.

[4] Z. Ebrahimi, B. Khaleghi, and H. Asadi, "PEAF: A power-efficient architecture for SRAM-based FPGAs using reconfigurable hard logic design in dark silicon era," *IEEE Trans. Comput.*, vol. 66, no. 6, pp. 982–995, Jun. 2017.

[5] M. Lin, A. El Gamal, Y.-C. Lu, and S. Wong, "Performance benefits of monolithically stacked 3-D FPGA," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 26, no. 2, pp. 216–229, Feb. 2007.

[6] P. Gaillardon *et al.*, "Connor, and G. De Micheli, "Design and architectural assessment of 3-D resistive memory technologies in FPGAs," *IEEE Trans. Nanotechnol.*, vol. 12, no. 1, pp. 40–50, Jan. 2013.

[7] G. Asadi and M. B. Tahoori, "Soft error rate estimation and mitigation for SRAM-based FPGAs," in *Proc. ACM/SIGDA 13th Int. Symp. Field-Program. Gate Arrays*, Feb. 2005, pp. 149–160.

[8] H. Asadi, M. B. Tahoori, B. Mullins, D. Kaeli, and K. Granlund, "Soft error susceptibility analysis of SRAM-based FPGAs in high-performance information systems," *IEEE Trans. Nucl. Sci.*, vol. 54, no. 6, pp. 2714–2726, Dec. 2007.

[9] E. Seevinck, F. J. List, and J. Lohstroh, "Static-noise margin analysis of MOS SRAM cells," *IEEE J. Solid-State Circuits*, vol. SSC-22, no. 5, pp. 748–754, Oct. 1987.

[10] J. S. Meena, S. M. Sze, U. Chand, and T.-Y. Tseng, "Overview of emerging nonvolatile memory technologies," *Nanosc. Res. Lett.*, vol. 9, no. 1, p. 526, Sep. 2014.

[11] H.-S. P. Wong *et al.*, "Metal–oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951–1970, Jun. 2012.

[12] W. Guan, S. Long, Q. Liu, M. Liu, and W. Wang, "Nonpolar nonvolatile resistive switching in Cu doped ZrO_2," *IEEE Electron Device Lett.*, vol. 29, no. 5, pp. 434–437, May 2008.

[13] Y. Y. Liauw, Z. Zhang, W. Kim, A. El Gamal, and S. S. Wong, "Nonvolatile 3D-FPGA with monolithically stacked RRAM-based configuration memory," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2012, pp. 406–408.

[14] W. Zhao, E. Belhaire, V. Javerliac, C. Chappert, and B. Dieny, "Evaluation of a non-volatile FPGA based on MRAM technology," in *Proc. IEEE Int. Conf. Integr. Circuit Design Technol.*, May 2006, pp. 1–4.

[15] P.-E. Gaillardon *et al.*, "Phase-change-memory-based storage elements for configurable logic," in *Proc. Int. Conf. Field-Program. Technol. (FPT)*, Dec. 2010, pp. 17–20.

[16] S. Yazdanshenas, B. Khaleghi, P. Ienne, and H. Asadi, "Designing low power and durable digital blocks using shadow nanoelectromechanical relays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 12, pp. 3489–3498, Dec. 2016.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.

[17] J. Cong and B. Xiao, "FPGA-RPI: A novel FPGA architecture with RRAM-based programmable interconnects," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 4, pp. 864–877, Apr. 2014.
[18] M. Liu and W. Wang, "rFGA: CMOS-nano hybrid FPGA using RRAM components," in *Proc. IEEE Int. Symp. Nanosc. Architect.*, Jun. 2008, pp. 93–98.
[19] P.-E. Gaillardon, M. H. Ben-Jamaa, G. B. Beneventi, F. Clermidy, and L. Perniola, "Emerging memory technologies for reconfigurable routing in FPGA architecture," in *Proc. 17th IEEE Int. Conf. Electron., Circuits, Syst. (ICECS)*, Dec. 2010, pp. 62–65.
[20] S. Paul, S. Mukhopadhyay, and S. Bhunia, "A circuit and architecture codesign approach for a hybrid CMOS–STTRAM nonvolatile FPGA," *IEEE Trans. Nanotechnol.*, vol. 10, no. 3, pp. 385–394, May 2011.
[21] P.-E. Gaillardon, D. Sacchetto, S. Bobba, Y. Leblebici, and G. De Micheli, "GMS: Generic memristive structure for non-volatile FPGAs," in *IEEE/IFIP 20th Int. Conf. VLSI Syst.-Chip, (VLSI-SoC)*, Oct. 2012, pp. 94–98.
[22] D. Suzuki *et al.*, "Fabrication of a 3000-6-input-LUTs embedded and block-level power-gated nonvolatile FPGA chip using p-MTJ-based logic-in-memory structure," in *Proc. Symp. VLSI Circuits (VLSI Circuits)*, Jun. 2015, pp. C172–C173.
[23] R. Rajaei, "Radiation-hardened design of nonvolatile MRAM-based FPGA," *IEEE Trans. Magn.*, vol. 52, no. 10, Oct. 2016, Art. no. 3402010.
[24] Y. Chen, J. Zhao, and Y. Xie, "3D-NonFAR: Three-dimensional non-volatile FPGA architecture using phase change memory," in *Proc. ACM/IEEE Int. Symp. Low Power Electron. Design*, Aug. 2010, pp. 55–60.
[25] O. Turkyilmaz *et al.*, "RRAM-based FPGA for 'normally off, instantly on' applications," in *Proc. IEEE/ACM Int. Symp. Nanosc. Architect. (NANOARCH)*, Jul. 2012, pp. 101–108.
[26] (2016). *Synopsys EDA Tools Flow*. [Online]. Available: http://www.synopsys.com/
[27] H. Li, P. Huang, B. Gao, B. Chen, X. Liu, and J. Kang, "A SPICE model of resistive random access memory for large-scale memory array simulation," *IEEE Electron Device Lett.*, vol. 35, no. 2, pp. 211–213, Feb. 2014.
[28] J. Luu *et al.*, "VTR 7.0: Next generation architecture and CAD system for FPGAs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 7, no. 2, Jun. 2014, Art. no. 6.
[29] S. Yang, "Logic synthesis and optimization benchmarks user guide version 3.0," Microelectron. Center North Carolina, Research Triangle Park, NC, USA, Tech. Rep., Jan. 1991.
[30] C.-Y. Wen *et al.*, "A non-volatile look-up table design using PCM (phase-change memory) cells," in *Proc. IEEE Symp. VLSI Circuits (VLSIC)*, Jun. 2011, pp. 302–303.
[31] A. Ahari, H. Asadi, B. Khaleghi, and M. B. Tahoori, "A power-efficient reconfigurable architecture using PCM configuration technology," in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, Mar. 2014, pp. 1–6.
[32] S. Tanachutiwat, M. Liu, and W. Wang, "FPGA based on integration of CMOS and RRAM," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 11, pp. 2023–2032, Nov. 2011.
[33] Y.-C. Chen, W. Wang, H. Li, and W. Zhang, "Non-volatile 3D stacking RRAM-based FPGA," in *Proc. 22nd Int. Conf. Field Program. Logic Appl. (FPL)*, Aug. 2012, pp. 367–372.
[34] P.-E. Gaillardon *et al.*, "A ultra-low-power FPGA based on monolithically integrated RRAMs," in *Proc. Design, Autom. Test Eur. Conf. Exhibit.*, Mar. 2015, pp. 1203–1208.
[35] A. Pirovano *et al.*, "Reliability study of phase-change nonvolatile memories," *IEEE Trans. Device Mater. Rel.*, vol. 4, no. 3, pp. 422–427, Sep. 2004.
[36] G. Lemieux and D. Lewis, *Design of Interconnection Networks for Programmable Logic*. New York, NY, USA: Springer, 2004.
[37] X. Tang, G. Kim, P.-E. Gaillardon, and G. De Micheli, "A study on the programming structures for RRAM-based FPGA architectures," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 4, pp. 503–516, Apr. 2016.
[38] X. Tang, G. De Micheli, and P.-E. Gaillardon, "A high-performance FPGA architecture using one-level RRAM-based multiplexers," *IEEE Trans. Emerg. Topics Comput.*, vol. 5, no. 2, pp. 210–222, Apr./Jun. 2016.
[39] X. Tang, E. Giacomin, G. De Micheli, and P.-E. Gaillardon, "Circuit designs of high-performance and low-power RRAM-based multiplexers based on 4T(ransistor)1R(RAM) programming structure," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 64, no. 5, pp. 1173–1186, May 2017.
[40] H. Michinishi, T. Yokohira, T. Okamoto, T. Inoue, and H. Fujiwara, "Testing for the programming circuit of LUT-based FPGAs," in *Proc. Asian Test Symp.*, Nov. 1997, pp. 242–247.
[41] *Virtex-6 FPGA Configurable Logic Block, User Guide*, Xilinx, San Jose, CA, USA, Feb. 2012.
[42] W. Kim *et al.*, "Forming-free nitrogen-doped AlO_X RRAM with sub-μA programming current," in *Proc. Symp. VLSI Technol. (VLSIT)*, Jun. 2011, pp. 22–23.
[43] (2016). *Nangate Open Cell Library*. [Online]. Available: http://www.nangate.com/
[44] M. Liu, W. Kuehn, Z. Lu, and A. Jantsch, "Run-time partial reconfiguration speed investigation and architectural design space exploration," in *Proc. Int. Conf. Field Program. Logic Appl. (FPL)*, Aug./Sep. 2009, pp. 498–502.
[45] C. Chiasson and V. Betz, "COFFE: Fully-automated transistor sizing for FPGAs," in *Proc. Int. Conf. Field-Program. Technol. (FPT)*, Dec. 2013, pp. 34–41.
[46] (2013). *Predictive Technology Model (PTM)*. [Online]. Available: http://ptm.asu.edu/
[47] R. Ho, "On-chip wires: Scaling and efficiency," Ph.D. dissertation, Dept. Elect. Eng., Stanford Univ., Stanford, CA, USA, 2003.
[48] J. Lamoureux and S. J. E. Wilton, "Activity estimation for field-programmable gate arrays," in *Proc. Int. Conf. Field Program. Logic Appl. (FPL)*, Aug. 2006, pp. 1–8.
[49] W.-C. Luo *et al.*, "Statistical model and rapid prediction of RRAM set speed–disturb dilemma," *IEEE Trans. Electron Devices*, vol. 60, no. 11, pp. 3760–3766, Nov. 2013.
[50] S. Yu, X. Guan, and H.-S. P. Wong, "On the stochastic nature of resistive switching in metal oxide RRAM: Physical modeling, Monte Carlo simulation, and experimental characterization," in *IEDM Tech. Dig.*, Dec. 2011, pp. 3–17.
[51] X. Tang, P.-E. Gaillardon, and G. De Micheli, "A high-performance low-power near-Vt RRAM-based FPGA," in *Proc. Int. Conf. Field-Program. Technol. (FPT)*, Dec. 2014, pp. 207–214.



Behnam Khaleghi received the B.Sc. and M.Sc. degrees in computer engineering from the Sharif University of Technology (SUT), Tehran, Iran, in 2013 and 2016, respectively. He is currently a Research Assistant with the Data Storage, Networks, & Processing (DSN) Laboratory, Department of Computer Engineering, SUT. He was a research assistant with the Chair for Embedded Systems, Karlsruhe Institute of Technology, from 2014 and 2015. His research interests include reconfigurable architectures and computer-aided design. He has two Best Paper Nominations at the DAC'17 and DATE'17.



Hossein Asadi (M'08–SM'14) received the B.Sc. and M.Sc. degrees in computer engineering from the Sharif University of Technology (SUT), Tehran, Iran, in 2000 and 2002, respectively, and the Ph.D. degree in electrical and computer engineering from Northeastern University, Boston, MA, USA, in 2007.

He was with EMC Corporation, Hopkinton, MA, USA, as a Research Scientist and a Senior Hardware Engineer, from 2006 to 2009. He has been with the Department of Computer Engineering, SUT, since 2009, where he is currently a tenured Associate Professor. He is the Founder and the Director of the Data Storage, Networks, and Processing Laboratory, the Director of the Sharif High-Performance Computing Center, the Director of the Sharif Information Technology Service Center, and the President of the Sharif ICT Innovation Center. His current research interests include data storage systems and networks, solid-state drives, operating system support for I/O and memory management, and reconfigurable and dependable computing.

Dr. Asadi was a recipient of the Technical Award for the Best Robot Design from the International RoboCup Rescue Competition, organized by AAAI and RoboCup, the Best Paper Award at the 15th CSI International Symposium on Computer Architecture and Digital Systems, the Distinguished Lecturer Award from SUT in 2010, and the Distinguished Researcher Award and the Distinguished Research Institute Award from SUT in 2016.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:54:37 UTC from IEEE Xplore. Restrictions apply.